QR Code and Barcode Detection System

1. Introduction

The objective of this project is to design and implement a system capable of detecting and decoding QR codes and barcodes from images using computer vision techniques. The system is developed using Python and OpenCV and supports both automated processing through a pipeline and interactive usage via a graphical interface. The main challenge lies in accurately detecting codes under different conditions such as noise, lighting variations, and partial occlusion.

2. Methodology (Pipeline)

The system is structured as a multi-stage pipeline, where each step contributes to improving detection accuracy and robustness.

2.1 Image Loading

Images are loaded using OpenCV (cv2.imread) or via an interactive interface. Each image is processed as a NumPy array in BGR format.

2.2 Preprocessing

Preprocessing is a critical stage that prepares the image for detection by enhancing relevant features and reducing noise. The following operations are applied:

-**Grayscale conversion**; Reduces computational complexity by removing color information while preserving intensity variations.
-Gaussian Blur: Smooths the image to reduce high-frequency noise and improve edge detection stability.
-**Otsu Thresholding:** Automatically determines an optimal threshold to convert the image into a binary representation, which is particularly useful for QR code detection.

-**Canny Edge Detection**: Extracts edges that are later used for contour-based detection of QR codes.

-**Sobel Gradient (Barcode-specific):** Computes directional gradients to emphasize vertical edges, which correspond to barcode structures.

-**Morphological Operations:** Applied to refine shapes:

- For QR: clean and connect edges.
- For barcode: group parallel lines into a single region

2.3 QR Code Detection

• **Candidate region detection**: Contours are extracted from the preprocessed binary image using cv2.findContours. Only contours with a sufficiently large area are kept to eliminate noise and irrelevant small regions.
• **Bounding box computation and ROI extraction**: For each selected contour, a bounding box is computed and used to extract a Region of Interest (ROI) from the original image. This step isolates potential QR code regions for further analysis.
• **Contrast enhancement**: The extracted ROI is converted to grayscale and enhanced using histogram equalization. This improves visibility and helps in detecting QR patterns under different lighting conditions.
• **Binarization (thresholding)**:An adaptive threshold (Otsu method) is applied to convert the ROI into a binary image. This highlights the black-and-white structure of QR codes, making them easier to decode.
• **Resolution improvement**: The ROI is resized (scaled up) to increase its resolution. This step improves decoding performance, especially for small or low-quality QR codes.
• **Multi-version decoding strategy**: The decoding process is applied on several versions of the ROI (original, grayscale, enhanced, thresholded, and resized). The system stops as soon as a valid decoding result is obtained, increasing robustness.
• **Result validation and visualization**: Once a QR code is successfully decoded, the extracted data is stored and the corresponding contour is drawn on the original image to indicate the detected region.

2.4 Barcode Detection

Barcode detection follows a structural approach that exploits the specific pattern of parallel lines in 1D barcodes.

• **Gradient computation (Sobel operator)**: The Sobel filter is applied to the image to compute gradients, particularly emphasizing vertical edges. This highlights the parallel line structure characteristic of barcodes, where intensity changes rapidly along one direction.
• **Thresholding (binarization)**: The gradient image is converted into a binary image using thresholding. This step simplifies the image by keeping only strong edge responses, making barcode regions more distinguishable from the background.
• **Morphological closing**: A morphological closing operation is applied to connect nearby edges. This fills gaps between barcode lines and merges them into a single continuous region, improving region detection.
• **Contour detection**: Contours are extracted from the processed image using cv2.findContours. These contours represent potential barcode regions.
• **Contour filtering**: Only contours with a sufficiently large area are kept. This removes small noisy regions and keeps only meaningful candidates that could correspond to barcodes.
• **Bounding box and ROI extraction**: For each valid contour, a bounding box is

computed and used to extract a Region of Interest (ROI) from the original image. This isolates the barcode for decoding.

• **ROI enhancement**: The ROI is converted to grayscale, enhanced using histogram equalization, and binarized using Otsu thresholding. This improves contrast and readability of barcode patterns.

• **Multi-version decoding strategy**: Decoding is attempted on several versions of the ROI (original, grayscale, enhanced, thresholded, resized). The process stops as soon as a valid result is found, increasing robustness.

• **Result validation and visualization**: Decoded barcode data is extracted and displayed, and the detected region is highlighted on the image using a bounding box.

2.5 Visualization

The final step consists of visualizing the results:

-QR codes are highlighted using polygons

-Barcodes are highlighted using bounding rectangles

-Decoded data is displayed alongside the image

-This allows easy verification of detection accuracy.

Figure 1:This figure shows the successful detection and decoding of an EAN-13 barcode. The system correctly identifies the barcode region using gradient-based edge detection (Sobel operator) and morphological processing. The bounding box accurately localizes the the barcode, and the decoding is performed using the pyzbar library.

Figure 2:This figure presents the detection of a CODE128 barcode. The system successfully extracts the region of interest (ROI) and decodes the embedded information. The use of thresholding and morphological closing helps in grouping the barcode lines into a single region

Figure 3: This figure shows a standard QR code detected using OpenCV's QRCodeDetector. The global detection method successfully identifies and decodes the QR code without requiring additional processing.

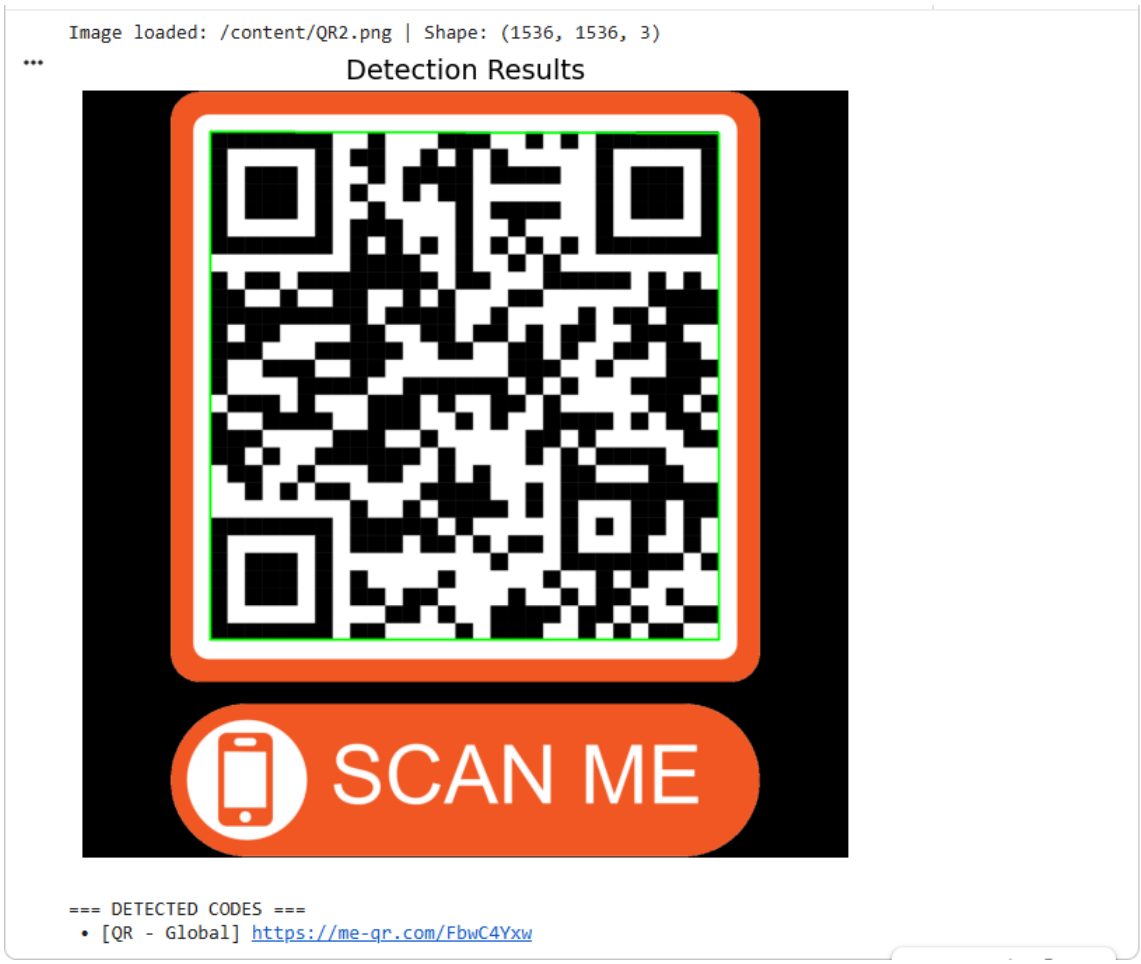

Figure 4: This figure illustrates the detection of a QR code with additional graphical elements (logo and "SCAN ME" label). Despite the added complexity, the system successfully detects and decodes the QR code.

3. OpenCV Functions and Concepts

This project relies on several key functions from OpenCV to perform image processing, feature extraction, and object detection.

• **Image conversion (cv2.cvtColor)**
This function is used to convert images from color (BGR) to grayscale. Grayscale simplifies processing by reducing the image to a single intensity channel, which is sufficient for edge detection and decoding.
• **Gradient computation (cv2.Sobel)**: The Sobel operator computes image gradients to detect intensity changes. It is particularly useful for highlighting edges, such as the vertical lines found in barcodes.
• **Thresholding (cv2.threshold)**: Thresholding converts grayscale images into binary images. Using Otsu's method allows automatic selection of the optimal threshold value, making the process adaptive to different lighting conditions.
• **Contour detection (cv2.findContours)**: This function detects continuous boundaries in a binary image. Contours are used to identify candidate regions that may contain QR codes or barcodes.
• **Contour analysis (cv2.contourArea)**: This function computes the area of a contour. It is used to filter out small regions and noise, keeping only significant shapes for further processing.
• **Bounding box extraction (cv2.boundingRect)**: This function computes a rectangular region around a contour. It is used to extract the Region of Interest (ROI) where a barcode or QR code is likely located.
• **Rotated rectangle (cv2.minAreaRect)**: This function computes the minimum-area rotated rectangle enclosing a contour. It is useful for detecting objects that are not perfectly aligned with the image axes.
• **Drawing functions (cv2.drawContours)**: These functions are used to visualize detection results by drawing contours or bounding boxes on the image.
• **Histogram equalization (cv2.equalizeHist)**: This technique enhances image contrast by redistributing pixel intensities. It improves visibility of patterns, especially in low-contrast regions.
• **Image resizing (cv2.resize)**: Resizing increases the resolution of small regions (ROI), improving the accuracy of decoding algorithms.

4. Challenges

Several practical challenges were encountered:

-Noise and blur affecting edge detection

-**Reflections on surfaces** (e.g., plastic bottles)
-**Broken or distorted barcode lines**
-**Sensitivity to preprocessing parameters** (kernel size, thresholds)
-**Large or imprecise ROI selection**, reducing decoding accuracy
-**Complex backgrounds** interfering with QR detection

Addressing these challenges required careful tuning of preprocessing and detection strategies

5. Conclusion

In conclusion, the project successfully demonstrates how classical computer vision techniques can be used to detect and decode QR codes and barcodes.

The combination of preprocessing, edge detection, morphological operations, and contour analysis provides a robust solution. Future improvements could include handling rotated images, improving detection in low-light conditions, and integrating deep learning techniques.